HybridGuard: Cross-Layer Fingerprint Consistency for Android Web Environments

Anonymous Author(s)

Abstract

Web services use browser-visible environment reports to recognize clients and inform security decisions, yet reports can be modified without corresponding device changes. In Android hybrid Web applications, Native, WebView Host, and App Web provide complementary observations of the same environment. Their semantic relationships offer potential integrity evidence: an individually plausible Web report may be incompatible with its system or container context.

HybridGuard associates observations from the same hybrid Web execution and checks their semantic compatibility, accounting for unavailable evidence and legitimate cross-API differences. We collect these observations and use controlled fingerprint manipulations and separate fixed-check comparisons to examine the effectiveness and limits of cross-layer references.

Across 14 controlled manipulation configurations, Full App detects 105/126 effective modifications (83.3%), with no alarms on 252 controlled normal records. A Native memory reference detects 18/18 modifications versus 6/18 for a global Web-only threshold, both with 0/48 normal alarms, showing the additional information supplied by device-specific context. A Native timezone reference and an absolute Web condition each detect 6/6 modifications, with 0/30 and 6/30 normal alarms, respectively; those six alarms arise from legitimate system timezone changes. In a separate WebView-host geometry study, a host reference and a fixed Web-height threshold each detect 6/6 modifications, with 0/66 and 6/66 normal alarms, respectively; the threshold flags legitimate layout changes. These findings support cross-layer consistency as useful integrity evidence when an observable, semantically related reference remains informative beyond the manipulated report. Neither Native nor Host is a root of trust, and consistency neither proves authenticity nor guarantees detection of coordinated cross-surface manipulation.

CCS Concepts

• **Security and privacy → Web application security**; *Intrusion/anomaly detection and malware mitigation.*

Keywords

browser fingerprinting, Android WebView, cross-layer consistency, manipulation detection, missing evidence

1 Introduction

Browser fingerprints summarize properties exposed by a client, including its user agent, language, timezone, display, and graphics environment. Web services use such observations to recognize clients or support authentication decisions, while the same mechanisms create privacy risks [1, 6, 8]. Their usefulness depends partly on the integrity of the reports: an attacker who controls a reporting surface may change what a service sees without changing the underlying device. Fingerprint spoofing and inconsistent evasive fingerprints are established problems [7, 10, 11].

Android hybrid applications make this problem distinctive. Web content executes inside a host application that can observe the operating system and configure a WebView. A JavaScript report is therefore one of several partially overlapping accounts of the same environment. An independent system browser can provide another account. This creates an opportunity to ask whether reports are mutually compatible, but also a risk of treating legitimate differences between APIs and containers as manipulation. The Web-security question we study is: *which cross-layer references help detect manipulated mobile Web reports, and what normal-alarm and missing-evidence costs do they introduce?*

A single-field rule illustrates the difficulty. A large reported memory value may exceed a particular device's plausible capacity while remaining below a global threshold. Conversely, a changed timezone may reflect an ordinary system-setting change rather than a manipulated webpage. A screen-height threshold may flag both an effective modification and a harmless layout expansion. The relevant evidence is often a relationship to a contextual reference, not whether an isolated value is unusual. Reference observations are themselves untrusted measurements: consistency cannot attest that all surfaces are truthful.

We develop HybridGuard to study this tradeoff. Its App collector records Android Native, WebView-host, and in-app Web observations under a fixed contract. Checks retain unknown and execution-failure states rather than silently assigning benign values. Offline selection combines admitted conditions into a compact detector subject to normal-alarm and definite-output constraints; online evaluation uses only the current observations required by the frozen detector. An optional pairing path links the App to an independent browser. We distinguish the full App detector, individual mechanism checks, and the retained App-plus-browser extension throughout.

Our evaluation has four parts. First, a controlled benchmark compares the full App detector with four retrained ablations and a small decision tree. Second, historical normal-device records expose the cost of optimizing only controlled detections. Third, memory, timezone, and geometry studies test concrete reference mechanisms against simpler conditions. Fourth, paired observations locate the incremental benefit of a cross-endpoint timezone check. The datasets and model configurations have development-use history; the study is a reproducible characterization of these materials, not a new blind test.

The contributions are:

- An explicit three-view cross-verification framework for Android Native, the WebView host, and in-app Web observations, with state-aware checks that separate contextual references, unavailable evidence, and manipulation alerts.

WWW '27 submission draft, 2027.

- A controlled component analysis: FULL APP flags 105/126 modifications with 0/252 controlled-normal alarms; disabling the historical-normal alarm cap adds three detections but produces alarms on all historical normal records.
- Mechanism and extension evidence that identifies both benefits and limits: device references recover missed memory modifications and tolerate tested normal changes, while the retained browser extension adds five timezone detections but leaves language modifications unresolved.

The findings support evaluating reference checks jointly with normal behavior and evidence availability. They do not establish a universally trusted observation layer, complete manipulation coverage, or a calibrated authentication policy.

2 Setting and Threat Model

2.1 Observation Surfaces

We distinguish four surfaces. *Android Native* exposes system and hardware-related measurements, such as memory and the system timezone. The *WebView host* controls the container, provider, bridge, and layout context. The *App Web runtime* exposes JavaScript APIs inside that WebView. An *external browser* runs a separate Web probe outside the application. Android documentation describes the host's ability to configure WebView behavior and the bridge between native code and JavaScript [2]; this arrangement does not make the native side an attested root of trust.

The App contract, APP177, comprises 84 Native, 26 host, and 67 in-app Web fields. A companion BROWSER67 contract contains 67 external-browser fields. A complete linked observation can be represented as PAIRED244. These names describe measurement contracts, not numbers of independently informative features or inputs used by every detector. In particular, the evaluated App candidate pool does not contain the new host-geometry check, and the retained browser extension uses one timezone relation rather than all 244 fields.

2.2 Adversary and Normal Behavior

The tested adversary changes selected client-visible reports through controlled tools, browser emulation, or runtime overrides. The benchmark includes changes to resource reports, user agent and platform, timezone, screen metrics, language and plugin surfaces, WebGL identifiers, and webdriver exposure. The attacker need not change the physical device. The evaluation labels require an observed effect at the target surface: an attempted operation with no measurable effect is retained separately.

A useful reference is a second observation that the tested modification has not changed in the same way. We do not assume that an adversary can never manipulate Native or host observations. An attacker able to coordinate every compared surface, modify the collector, or compromise the collection backend may evade consistency checks. The study neither proves resistance to that attacker nor establishes that independently launched endpoints are statistically or administratively independent.

Normal behavior includes changes to system timezone or language, display orientation, and application layout. APIs can also reduce precision, lack support, return different units, or expose container-specific state. Such variation must be represented in applicability checks and normal controls rather than labeled malicious because two values differ. A manipulation alert indicates a violated tested condition, not a finding of fraud, account takeover, or malicious user intent.

2.3 Evidence Units and Decision States

The unit of evaluation is a saved *record position*, or an App–browser paired position where required, with an independently assigned experiment role. Counts of repeated operations are not counts of independent devices. A detector returns alert (T), no alert (F), or unknown (U); execution failure and an empty model are separate outcomes. An unavailable operand may prevent a conclusion, and a no-alert result establishes only that the selected conditions did not trigger. Ground-truth experiment roles and operation records are used for evaluation, not as detector inputs.

3 HybridGuard

3.1 Collection and State-Aware Checks

Figure 1 separates measurement, offline selection, and current-record evaluation. Each field carries a value and an observation status. Normalization handles units and schema versions before a check executes; unsupported, missing, or unparsable operands remain unavailable. Original records, receipts, and pairing metadata support source attribution. A paired view requires two linked observations; an incomplete browser capture leaves an App-only record rather than a fabricated browser vector. Pair identifiers, experiment labels, environment names, and stage metadata are excluded from prediction inputs.

A check is a predicate together with its prerequisites and interpretation. Candidate conditions include single-field thresholds, relations within the App Web view, and Native–Web references. A reference is not an instruction to require exact equality: measurements may be bucketed, unit-converted, or tied to a collection interval. Documentation informs these interpretations, while project predicates and tolerances remain research choices rather than official attack rules.

Let $r_j(x) \in \{\mathsf{T}, \mathsf{F}, \mathsf{U}, \mathsf{FAILED}\}$ be the state of a selected condition on current observation x. The detector is an OR of selected singleton conditions. After checking selected-input validity, any execution or binding failure yields FAILED; otherwise any T yields an alert, all F yields no alert, and the remaining cases yield U. An empty selected set has its own status. Thus a known violation can establish an alert despite another unknown condition, whereas unknown evidence cannot establish a definite no-alert result. Unselected fields do not independently cause failure.

3.2 Device References

Memory. The memory reference tests whether a positive Web-reported memory value exceeds a coarse upper envelope of observed Native memory:

$$r_{\mathrm{mem}}(x) = \left[m_{\mathrm{web}} > 2^{\lceil \log_2 m_{\mathrm{native}} \rceil} \right]. \tag{1}$$

Both operands are expressed in GiB, and validity checks include a Native value of at least 0.25 GiB. The envelope accommodates coarse reporting instead of demanding physical-memory equality. It

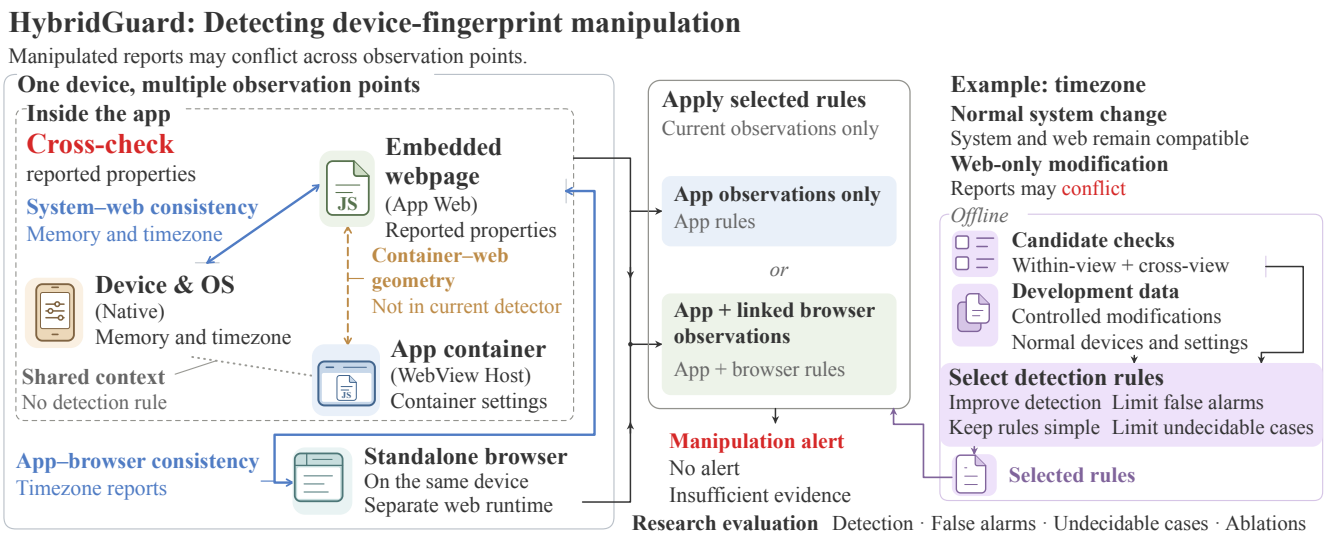


Figure 1: HybridGuard uses complementary observations from the same device to check fingerprint reports. Offline selection constrains normal alarms and definite-output coverage; frozen detectors consume current App observations or linked App–browser observations. Solid comparison edges denote detector relations. Host–Web geometry is a separate study; the dotted Native–Host edge supplies context only. No surface is trusted ground truth. No alert does not establish safety, and execution failures are recorded separately.

targets upward violations; a smaller or coordinated plausible report can evade it. We compare it with the fixed condition $m_{\text{web}} > 8$ in a separate mechanism study.

Timezone. The Native timezone reference derives the effective offset from the device's IANA timezone and its collection interval, using the fixed timezone database 2026c. It compares this offset with the Web `getTimezoneOffset` value after reversing the UTC-offset sign convention. It compares effective offsets, not timezone-name strings. An ambiguous transition interval or unusable prerequisite yields unknown; zero and minus one are not automatically missing sentinels. This permits an ordinary system timezone change when both observations move consistently, while exposing a Web-only override that does not match the Native reference.

Host geometry. A separate check compares visual-viewport geometry with a host measurement of the containing View. For either axis, let P be the CSS visual-viewport size multiplied by device-pixel ratio and visual-viewport scale, and let H be the host View size less its own padding. The check alerts if $P > H + \epsilon$, with $\epsilon = 2 + 4 \cdot 2^{-23} \max(|P|, |H|, 1)$ pixels. Snapshot binding and host-before/after stability are prerequisites. The implemented domain also requires an attached, visible host View, zero padding, and an identity View transform; observations outside that domain yield unknown. This fixed engineering tolerance is not a platform guarantee. The condition is evaluated as a specialist mechanism and is *not* part of Full App.

3.3 Constrained Detector Selection

The App pool contains 50 base templates and three fixed relations, expanded using the training-side encoder and admissible polarities. The full pool has 65, 65, and 56 expanded atoms across the three model configurations. A candidate must be sufficiently assessable on historical normal training data; absence of a comparable field is not resolved by assuming that the condition is false.

For a selected set S, the sparse stage maximizes the heuristic objective

$$J(S) = \frac{1}{K} \sum_{k=1}^{K} \bar{d}_k(S) - 0.005\, C(S), \tag{2}$$

where $K = 14$, $\bar{d}_k$ averages detection over training environments and then effective modification positions within configuration k, and C counts clauses plus literal occurrences. Selection uses greedy improving additions, retains the best feasible prefix, and performs a backward-pruning pass; it does not guarantee a global optimum.

Feasibility limits a detector to eight singleton OR clauses, complexity 16, and two clauses per registered family, without opposite polarities of the same atom. Separate training-normal alarm budgets are $\lfloor 0.05 \cdot 168 \rfloor = 8$ for controlled records and $\lfloor 0.05 \cdot 630 \rfloor = 31$ for historical cloud-device records. Candidate assessability and final definite-output coverage on the latter must each reach 90%; controlled coverage is constrained per phase at 80%. Fitting disallows execution failures and uses a 60-second deadline per fit.

The subsequent retention stage starts from that configuration's sparse solution. It can add semantic diversity without losing training detections, or replace a same-group condition with a registered higher-quality alternative under per-record nonregression checks. It does not continue optimizing Equation 2. We evaluate the saved retention outputs consistently, rather than choosing a stage separately for each result cell. The resulting detector is frozen before its designated evaluations.

3.4 Paired Browser Extension

The retained paired detector adds a cross-endpoint timezone condition, C1, to the frozen App detector. C1 alerts when the linked App Web and external-browser offsets are unequal. Both are finite numeric Web offsets with the same sign convention; zero and minus one are valid. C1 has no timestamp gate, unlike the interval-aware Native timezone reference. Missing or inapplicable evidence remains unknown under the same state-aware combination. The extension selection compared the empty set, C1, a language condition C2, and their combination, with separate normal-alarm budgets of 31/630 MTC training records, 0/12 pilot normals, and 1/34 matched normals, plus 90% definite-output coverage. C2 alarms on two matched normal preference changes, and the two-condition combination provides only 566/630 definite MTC training outputs; neither meets the frozen constraints. This offers a second endpoint for testing a mismatch; it does not establish which endpoint is truthful or make language differences suspicious by definition. The extension is evaluated separately from the App benchmark because the original App positions do not have contemporaneous external-browser counterparts.

4 Evaluation

We ask four questions: which modifications are detected and which components matter (RQ1); what cost is hidden by controlled-normal results (RQ2); how contextual references change individual checks (RQ3); and where an external browser adds capability (RQ4).

4.1 Materials and Protocol

Controlled App benchmark. The benchmark contains 378 positions: 14 modification configurations, three repetitions, three Android emulator environments (API 29, 30, and 36 with SwiftShader), and three stages per run. Admitted runs contain a normal pre-position, an effective modification, and a confirmed normal post-position. This gives 126 effective modifications and 252 normal records. The normal post-position is an evaluation/control record, not an input available to an earlier prediction. Three leave-one-environment-out configurations each use 252 training positions (84 modified, 168 normal) and 126 held-out positions (42 modified, 84 normal). We aggregate the disjoint held-out positions once. Development exposure to these environments and recipes means this partition is not evidence of unseen-tool or independent physical-device generalization.

Historical normal records. The mobile test-cloud (MTC) material comprises 891 representative records: 630 discovery records for constraints and tree training, 144 historical development records, and 117 historically reserved evaluation records. Representatives are manufacturer/model/Android-release combinations, not 891 independent physical devices; model families and known installation/session associations constrain grouping. The last two sets are reported separately. Each frozen model is applied to the same records; three model outputs do not triple the sample size. "Normal" means no study-defined target manipulation, not an attestation that a cloud device has no automation or debugging software.

Specialist and paired material. The memory study has 18 effective modifications and 48 normal records, plus six operations with no observable effect. The timezone study has six modifications and 30 normal records; six normal system changes are a subset of the 30. The screen study has six modifications and 66 normal records, including six normal layout expansions. The App–browser language/timezone material contains 60 paired positions: 14 effective modifications and 46 normal records, from an 18-position pilot and a 42-position matched batch. Normal records total 12 + 34, not just the matched subset. These materials have development-use history and distinct roles. We do not add their denominators to produce an overall accuracy or an independent population sample.

Methods. Full App denotes the saved App retention detector. Four ablations are retrained from their own sparse stage: remove the MTC alarm cap, use App-Web-only candidates, remove the memory reference, or remove the timezone reference. Removing the MTC cap raises its allowance to 630, while retaining MTC data, assessability and coverage constraints, and the controlled-normal budget. Web-only removes Native dependencies, so it changes available information as well as the relationships. The host-geometry specialist and Browser C1 are outside this pool.

The App tree uses the same fold-specific admitted candidate representation, with separate true/false/unknown indicators, maximum depth three, minimum leaf size two, Gini splitting, and a fixed random seed. Training weight is one half for effective modifications, one quarter for controlled normals, and one quarter for MTC normals; modification weight is balanced across configurations. This is a fixed small-capacity reference, not the performance ceiling of machine learning or a classifier over all raw App fields. Rule rows use the saved `RETENTION` stage and tree rows the saved `FITTED` stage.

Measures and reporting. A detection is an alert on an effective modification; a normal alarm is an alert on a normal record. We retain no-alert, unknown, failed, and empty-model counts. Unknown records stay in denominators rather than being counted as benign. Percentages are descriptive ratios for the stated cohort; we do not estimate population recall, claim significance from repeated configurations, or invent confidence intervals from dependent positions. The tables transcribe frozen results; assembling this draft involved no new collection, fitting, selection, prediction, or timing.

4.2 RQ1: App Detection and Components

Full App detects 105/126 modifications (83.3%). It produces no alarms on the 252 controlled normal records. Table 1 compresses the 14 configurations into five groups without dropping configurations: group membership requires matching complete outcome-count vectors across all six methods. Counts within a row are per original configuration, and the group-size column preserves the weighting. These are count equivalences, not claims of identical individual outputs across recipes.

The resource modification accounts for the nine detections lost by Web-only and by removing the memory reference: both fall to 96/126. The small tree detects 72/126. Its additional misses include the tested timezone and plugin/MIME modifications as well as resource and screen modifications. This comparison identifies the behavior of the fixed representation and capacity; it does not show that rules generally outperform learned classifiers.

Table 1: App-side detection and component comparisons. Cells give detections out of nine effective modifications per original configuration; parentheses give differences from Full App. Fourteen configurations are grouped only when their complete six-method state-count vectors agree. Adjacent equal results are merged. Group sizes retain the original weighting.

Modification group	Group size	Full App	No MTC alarm cap	App Web only	No memory relation	No timezone relation	App tree
CPU + memory	1	9		0 (−9)		9	0 (−9)
Screen metrics	1	6	9 (+3)	6			0 (−6)
Timezone; plugins/MIME	2	9					0 (−9)
Shared detections†	8	9 (all detected)					
Webdriver; WebGL IDs	2	0 (all missed)					
Total detections	14	105/126	108/126	96/126	96/126	105/126	72/126

Gray, green and red indicate equal, more and fewer detections than Full App, respectively; colors do not rank overall method quality. †Playwright, Puppeteer, CDP multi-field and Stealth presets; platform, UA, UA + platform and language-list modifications. All methods have 0/252 controlled-normal alarms; unknown, failed and empty-model counts are zero. Rules use saved RETENTION results and the tree uses FITTED results. The extra three screen detections without the cap have a separate normal-record cost (Table 2).

All methods miss the nine webdriver-only and nine WebGL-identifier modifications. In the current historical-data-compatible pool, raw WebGL unavailability and loss of the original webdriver-false semantics prevent corresponding candidates from meeting assessability requirements. The result is a coverage limit of the admitted representation, not proof that these modifications are intrinsically undetectable.

Removing the MTC alarm cap increases screen detections from six to nine and the total to 108/126. Removing the timezone reference leaves the total at 105/126 because an absolute offset condition substitutes on this benchmark. Neither observation is enough to choose a method: the first requires inspecting normal-record costs, and the second requires normal-setting changes.

4.3 RQ2: The Cost of Removing Normal Constraints

Table 2 shows why the extra three detections do not justify the cap-free variant. With the cap, model configurations 01/02 generate one alarm in each historical evaluation cohort; configuration 03 generates none. Removing the cap alarms on all 144 and all 117 records for every configuration. Both methods still generate 0/252 alarms in the controlled benchmark, so that normal set alone does not expose the failure.

The cap-free models can select broad conditions such as hardware concurrency greater than one or a language-list length greater than one. These happen to respect the controlled-normal allowance while being common on the MTC records. The failure therefore concerns the support of the normal distribution, not only a different threshold along a well-calibrated tradeoff curve. The 630 training records tell the same story: with the cap, alarms are 6/6/0 and unknowns are 51 in each configuration; without it, all 630 alarm.

Low alarm counts must also be read alongside unknowns. FULL APP has 8/144 and 6/117 unknown outcomes, or 14/261 within each configuration. Removing its memory reference reduces the combined unknown count to 1/261 but loses resource detections. The small tree gives binary outputs even when unused candidate measurements are missing; that behavior does not recover those measurements. We thus report decision coverage as part of the result, rather than equating no recorded alarm with usable normal evidence.

4.4 RQ3: What Device References Change

Table 3 compares *fixed individual checks*, not complete-detector ablations. The three studies answer different questions and retain separate denominators.

Memory. The device-referenced bound detects all 18 effective memory modifications, while the Web-only > 8 GiB check detects six. The additional 12 are the tested upward changes to 4 and 8 GiB; both conditions detect the six changes to 16 GiB. Both have 0/48 normal alarms. Six operations targeting 2 GiB have no observable effect and are excluded from the effective-modification denominator; neither check alarms on them. These results support the local value of a device-dependent bound, without showing sensitivity to downward, within-envelope, or coordinated changes.

Timezone: distinguish system changes from report changes. Both the Native-reference check and the absolute-offset check detect all six tested Web timezone modifications. Their normal alarms differ: zero versus six out of 30. The six alarms come from legitimate system changes. This explains why an unchanged aggregate score after removing the timezone relation does not imply that the relation is redundant. In the full-model replay, the no-timezone and Web-only ablations also alarm on those six normal changes, whereas FULL APP does not.

Screen geometry: retain detection without a layout shortcut. The fixed Web-height threshold detects six modifications but also alarms on six of 66 normal records. A same-Web viewport relation gives no normal alarms but misses all six modifications. The host bound detects all six with 0/66 normal alarms, distinguishing the tested layout expansion from the tested report modification. This is a standalone result: FULL APP configurations 01/02 still select the older height condition and alarm on those six normal layout changes; configuration 03 detects neither the changes nor the six modifications. We do not substitute the new condition's result for an unmeasured integrated-detector improvement.

Table 2: Normal-record outcomes with and without the MTC alarm cap. Removing the cap makes every record in both historical evaluation cohorts trigger an alarm, while controlled detections increase only from 105/126 to 108/126. Both methods still have 0/252 controlled-normal alarms. Counts are records, with unknown outcomes reported separately.

Historical cohort	Model configuration	With cap (Full App)			Without cap		
		Alarm	No alarm	Unknown	Alarm	No alarm	Unknown
Evaluation ($N = 144$)	01/02	**1**	135	8	**144**	0	0
	03	**0**	136	8			
Reserved ($N = 117$)	01/02	**1**	110	6	**117**	0	0
	03	**0**	111	6			

Equal counts are merged for display; each configuration uses the same records, so denominators are not pooled. Failed and empty-model counts are zero. The separate 630 constraint-training records yield 6/6/0 alarms and 51 unknowns per capped configuration (01/02/03), versus 630 alarms and no unknowns without the cap. These historical cohorts are not a new blind evaluation.

Table 3: Specialist comparisons of fixed checks. Device-reference conditions (*) supplement detection or tolerate normal changes in these separate cohorts. Entries are detection and normal-alarm counts; the denominators appear once per group. These are individual checks, not complete-model ablations.

Specialist cohort	Fixed condition	Detected	Normal alarms
Memory	Web memory > 8 GiB	6	0
18 modifications; 48 normal	System-memory upper bound*	18	0
Timezone	Absolute timezone offset	6	6
6 modifications; 30 normal	System-timezone reference*	6	0
Screen	Fixed Web-height threshold	6	6
6 modifications; 66 normal	Same-Web viewport relation	0	0
	Host-geometry upper bound*	6	0

Six memory operations with no observable effect produce no alarms and are excluded from the 18 effective modifications. Normal timezone changes (6) and layout changes (6) are subsets of their 30 and 66 normal records. Unknown and failed counts are zero. The host-geometry check remains a standalone specialist condition; it is not integrated into Full App.

4.5 RQ4: Where the External Browser Helps

On the 60 paired positions, Full App detects the two App timezone modifications, but neither App language modification nor any Browser modification. Adding the retained C1 relation detects all five Browser timezone modifications, yielding 7/14 instead of 2/14 (Table 4). The five new detections belong to one family and one endpoint. The two App-language and five Browser-language modifications remain undetected; C1 is not a general cross-endpoint manipulation detector.

Both methods have 0/46 normal alarms in this cohort, and the displayed counts agree across the three frozen configurations. They are displayed once, not summed to 21/42 detections or treated as independent replications. Historical normal evaluation supplies a further cost: within each configuration, adding C1 preserves the alarm count but increases unknowns from 14 to 17 out of the same combined 261 records. The two original historical groups remain distinct in the saved data. This targeted gain consequently neither proves complete evidence coverage nor demonstrates a universal zero-alarm rate.

5 Discussion, Limitations, and Ethics

What the evidence establishes. Cross-layer references can add information that selected single-field checks lack, and can also explain why an ordinary setting change should not alarm. Their value is conditional on available, correctly interpreted measurements. The strongest conclusion is therefore about specific mechanisms and their tradeoffs. The observations do not establish an independently trusted layer or a theoretical guarantee of detecting inconsistent manipulation.

Generality and attribution. The controlled benchmark covers three emulator environments and a finite set of recipes. Its repeated stages and development exposure limit independence. MTC normal records broaden the observed settings but do not supply attacks on an independent sample of physical devices. They are not clean-device attestations, and their historical split names do not erase development exposure. An ablation retrains the remaining pool, so its difference measures the behavior of that entire constrained alternative; it is not a causal estimate for arbitrary devices. The tree comparison controls representation and capacity but is not a comparison with tuned modern classifiers. No adaptive adversarial optimization or calibrated operational risk assessment is evaluated.

Missingness, drift, and integration. Unknown output is an operational cost, not a successful rejection or a harmless missing column. Adding a reference can turn a definite no-alert result into unknown, as seen with C1. Changes in WebView providers, API exposure, locale conventions, or device coverage may invalidate admissibility assumptions. The geometry check needs integrated selection and

Table 4: Increment from the retained cross-endpoint timezone check C1. All five additional detections are Browser timezone modifications; App timezone detection is preserved and both language families remain undetected. Bold marks changed counts, not statistical significance.

Method	App modifications		Browser modifications		Total detected	Normal alarms
	Timezone	Language	Timezone	Language		
Full App	2/2	0/2	0/5	0/5	2/14	0/46
Full App + C1	2/2	0/2	**5/5**	0/5	**7/14**	0/46
Count difference	0	0	**+5**	0	**+5**	0

These 60 paired positions have development-use history (14 effective modifications, 46 normal); unknown and failed counts are zero. Counts agree across three frozen configurations and are shown once, without pooling or claiming equal record-level outputs. On the separate historical MTC cohorts (144 and 117 records), alarms are unchanged within each configuration while combined unknowns increase from 14 to 17 out of 261.

real-device validation, including window modes, keyboard interaction, and padding cases. C1 has no timestamp gate: collection-time differences can be benign, so its observed 0/46 normal alarms do not establish general robustness to asynchronous setting changes. Neither all 177 App fields nor all 244 paired fields have demonstrated marginal utility.

Privacy and responsible use. Collecting more device information to assess integrity can itself enable tracking. WebView tracking research and W3C guidance make that cost explicit [12, 13]. The reported manipulations concern controlled emulator environments; historical normals come from a cloud-device collection process. Its documented App permission scope is network access and network-state observation, without requesting contacts, SMS, photos, location, camera, or microphone access. This bounds that collection request, not all possible privacy implications of the fingerprint.

The draft artifact provides derived counts, condition definitions, and table sources; it does not bundle raw device payloads, pair tickets, or persistent identifiers. Institutional review or exemption and participant-consent approval are not established by the examined study artifacts, and we make no such claim. Those requirements and any raw-data release basis must be resolved through the applicable institutional process. A future deployment should minimize retained information and provide a fallback for unknown or conflicting evidence; these results do not justify automatic denial of service or authentication based solely on a fingerprint alert.

6 Related Work

Fingerprinting and authentication. Laperdrix et al. survey browser fingerprinting attributes, uses, and defenses [6]. Alaca and van Oorschot analyze device fingerprinting as an additional authentication signal [1], while Spooren et al. identify limitations of mobile fingerprints for risk-based authentication [8]. Cao et al. study cross-browser identification using OS- and hardware-level features [3]. These works motivate inspecting relationships among reports, but linking an identity across browsers is different from deciding whether a current observation violates a consistency condition. HybridGuard reports integrity evidence, not identity uniqueness or authentication success.

Spoofing and inconsistency analysis. Gummy Browsers demonstrates targeted browser spoofing [7]. Fp-Scanner examines inconsistencies introduced by fingerprinting countermeasures [10]; Him of Many Faces characterizes adversarial and benign fingerprints from commercial websites [14]. FP-Inconsistent studies spatial and temporal inconsistencies in evasive bot traffic [11]. BrowserFM represents dependencies among browser configurations and fingerprint features [5]. Thus inconsistency detection and structured fingerprint relationships are established ideas. Our novelty claim concerns the explicit formulation of cross-verification across Native, host, and in-app Web observations, with explicit missing states, constrained selection, and controlled normal-change comparisons. Different tasks, labels, inputs, and datasets prevent a meaningful numerical comparison of their published performance with our modification counts; we do not claim empirical superiority over these systems.

Android WebViews and boundary crossing. Tiwari et al.'s preprint examines Android WebView fingerprinting [9]. Tracking Without Borders studies WebViews' role in bridging mobile and Web tracking [13]. Cross-Boundary Mobile Tracking observes Java-to-JavaScript information diffusion in WebViews [4]. These studies show why an embedded Web runtime should be considered with its application context. Our question concerns whether complementary measurements expose controlled report modifications while tolerating ordinary changes. We evaluate host geometry separately and do not infer complete three-layer detector contributions merely from collecting all three surfaces.

7 Conclusion

HybridGuard studies fingerprint manipulation through complementary device observations, constrained rule selection, and explicit evidence states. On the saved controlled App benchmark, it detects 105/126 effective modifications with 0/252 controlled-normal alarms. Historical normals expose a severe cost hidden by the cap-free variant's three extra detections. Specialist checks show how references can recover memory modifications and tolerate tested normal changes; the paired extension contributes five Browser timezone detections. Shared WebGL/webdriver misses, language misses, unknown outcomes, and unintegrated host geometry remain material boundaries. The results support targeted reference checks with normal and missing-evidence accounting, while leaving independent generalization and operational deployment to further evaluation.

References

[1] Furkan Alaca and Paul C. van Oorschot. 2016. Device Fingerprinting for Augmenting Web Authentication: Classification and Analysis of Methods. In *Proceedings of the 32nd Annual Conference on Computer Security Applications*. ACM, 289–301. doi:10.1145/2991079.2991091

[2] Android Developers. 2026. Build Web Apps in WebView. Living documentation; accessed 17 October 2026. https://developer.android.com/develop/ui/views/layout/webapps/webview

[3] Yinzhi Cao, Song Li, and Erik Wijmans. 2017. (Cross-)Browser Fingerprinting via OS and Hardware Level Features. In *Network and Distributed System Security Symposium*. doi:10.14722/ndss.2017.23152

[4] Sohom Datta, Michalis Diamantaris, Ahsan Zafar, Junhua Su, Anupam Das, Jason Polakis, and Alexandros Kapravelos. 2026. Cross-Boundary Mobile Tracking: Exploring Java-to-JavaScript Information Diffusion in WebViews. In *Network and Distributed System Security Symposium*. doi:10.14722/ndss.2026.230910

[5] Maxime Huyghe, Clement Quinton, and Walter Rudametkin. 2025. BrowserFM: A Feature Model-Based Approach to Browser Fingerprint Analysis. In *NDSS Workshop on Measurements, Attacks, and Defenses for the Web*. doi:10.14722/madweb.2025.23017

[6] Pierre Laperdrix, Nataliia Bielova, Benoit Baudry, and Gildas Avoine. 2020. Browser Fingerprinting: A Survey. *ACM Transactions on the Web* 14, 2, Article 8 (2020), 33 pages. doi:10.1145/3386040

[7] Zengrui Liu, Prakash Shrestha, and Nitesh Saxena. 2022. Gummy Browsers: Targeted Browser Spoofing Against State-of-the-Art Fingerprinting Techniques. In *Applied Cryptography and Network Security (Lecture Notes in Computer Science, Vol. 13269)*. Springer, 147–169. doi:10.1007/978-3-031-09234-3_8

[8] Jan Spooren, Davy Preuveneers, and Wouter Joosen. 2015. Mobile Device Fingerprinting Considered Harmful for Risk-Based Authentication. In *Proceedings of the Eighth European Workshop on System Security*. ACM, Article 6, 6 pages. doi:10.1145/2751323.2751329

[9] Abhishek Tiwari, Jyoti Prakash, Alimerdan Rahimov, and Christian Hammer. 2022. Our Fingerprints Don't Fade from the Apps We Touch: Fingerprinting the Android WebView. Preprint. arXiv:2208.01968 [cs.CR] doi:10.48550/arXiv.2208.01968

[10] Antoine Vastel, Pierre Laperdrix, Walter Rudametkin, and Romain Rouvoy. 2018. Fp-Scanner: The Privacy Implications of Browser Fingerprint Inconsistencies. In *27th USENIX Security Symposium*. USENIX Association, 135–150. https://www.usenix.org/conference/usenixsecurity18/presentation/vastel

[11] Hari Venugopalan, Shaoor Munir, Shuaib Ahmed, Tangbaihe Wang, Samuel T. King, and Zubair Shafiq. 2025. FP-Inconsistent: Measurement and Analysis of Fingerprint Inconsistencies in Evasive Bot Traffic. In *Proceedings of the 2025 ACM Internet Measurement Conference*. ACM, 134–149. doi:10.1145/3730567.3732919

[12] W3C Privacy Working Group. 2025. *Mitigating Browser Fingerprinting in Web Specifications*. W3C Group Note. World Wide Web Consortium. Editors: Nick Doty and Tom Ritter. Published 25 September 2025; accessed 17 October 2026. https://www.w3.org/TR/fingerprinting-guidance/

[13] Nipuna Weerasekara, Jose Miguel Moreno, Srdjan Matic, Joel Reardon, Juan Tapiador, and Narseo Vallina-Rodriguez. 2025. Tracking Without Borders: Studying the Role of WebViews in Bridging Mobile and Web Tracking. *Proceedings on Privacy Enhancing Technologies* 2025, 4 (2025), 745–762. doi:10.56553/popets-2025-0155

[14] Shujiang Wu, Pengfei Sun, Yao Zhao, and Yinzhi Cao. 2023. Him of Many Faces: Characterizing Billion-Scale Adversarial and Benign Browser Fingerprints on Commercial Websites. In *Network and Distributed System Security Symposium*. doi:10.14722/ndss.2023.24394

A Configuration and Artifact Details

This appendix documents display compression and reproducibility details. The main text contains all results needed for its conclusions. The companion source package includes the four English table sources, their saved aggregate data, a source map, and a count-verification script. The script checks saved counts and compression; it does not rerun the detector or establish new experimental validity.

A.1 The Fourteen Configurations

Table 1 preserves five groups. The single-configuration groups are CPU plus memory and screen metrics. The two-configuration group with additional tree misses consists of timezone-only and plugin/MIME modifications.

The eight configurations detected by all methods are four legacy tool presets (Playwright, Puppeteer, CDP multi-field, and Stealth), platform-only, user-agent-only, user agent plus desktop platform, and the language-list preset. The common-miss group contains webdriver-only and WebGL-identifier modifications. A language-list modification here is not interchangeable with the App/browser language operations in Table 4. The companion aggregate-data mapping preserves the exact original recipe identifiers, suffixes, and source rows.

Group sizes are 1, 1, 2, 8, and 2. Multiplying each row's per-configuration detection count by its group size and summing gives 105, 108, 96, 96, 105, and 72 detections for the six methods. Each original configuration contributes nine effective modifications and 18 normal records. The compression tests the full count vector (N, T, F, U) together with failure and empty-model counts, not only the detection count. It preserves a mapping to every original configuration.

A.2 Frozen Models and Comparison Roles

The three App configurations hold out the API 29, API 30, and API 36 emulator environments, respectively. `APP_FULL` denotes the retained `B_REL_TZ` models; `A_NO_MTC_CAP`, `A_APP_WEB_ONLY`, `A_NO_MEMORY_REL`, and `A_NO_TIMEZONE_REL` denote retrained alternatives. `APP_TREE` is the candidate-state tree, and `PAIRED_BASE` is the frozen App detector plus C1. No cell substitutes a specialist condition or another model's best result.

The tree implementation uses scikit-learn 1.7.2, `splitter=best`, `random_state=20261006`, no class-weight override, and a positive-class probability threshold of 0.5. Its fold-specific inputs contain 63, 63, and 54 admitted atoms, each with three state indicators. These counts differ from the full expanded pool because training-side admission is applied. Fifteen positions per controlled held-out set and 73 of the 891 MTC records contain an unknown candidate; the saved decision paths do not use those missing atoms. A binary output is therefore not proof of complete observation.

A.3 Conditions, Unknowns, and Study Boundaries

The memory comparison uses the Native envelope and the fixed Web-memory threshold at 8 GiB. The absolute timezone comparator alerts above −480 minutes; the Native reference uses effective IANA offsets with fixed timezone data. The old screen comparator uses Web inner height greater than 710 CSS pixels. The host condition applies snapshot stability and the tolerance in Section 3; its measured result does not imply integration into the App model.

C1 requires both its field-status and field-quality markers to indicate an observed value. It accepts finite numeric offsets from the linked App Web and browser records, including zero and minus one, and alerts on inequality. Both offsets use the same Web sign convention. It has no timestamp acceptance gate. Pair integrity errors yield failure; absent, nonnumeric, nonfinite, or nonobserved operands yield unknown. The model's selected-input failure precedence and OR semantics still apply.

Across the two historical evaluation groups, Full App's per-configuration totals are $(T, F, U) = (2, 245, 14)$ for configurations 01/02 and $(0, 247, 14)$ for 03. With C1, they become $(2, 242, 17)$ and $(0, 244, 17)$, respectively. These are sums of the 144- and 117-record groups within one model, not additional records. The six no-effect memory operations remain outside effective-modification counts. Newer geometry engineering-smoke positions and later resource-extension experiments are outside the four selected materials and are not pooled into the manuscript results.

A.4 Training-Side Admission and Extension Selection

Numeric threshold expansion uses training-only quantiles at 0.25, 0.5, and 0.75. Candidate support requires at least three complete true/false triplets and at least two triggered modification triplets in at least one bundle/environment, in addition to the normal assessability criteria. These thresholds and the normal budgets are fixed before scoring; historical evaluation records do not supply threshold quantiles.

The browser-extension comparison enumerates four subsets: no addition, C1, C2, and C1 plus C2. Feasible sets are ranked by the equal mean of detection ratios for App timezone, App language, Browser timezone, and Browser language modifications, then by fewer additions and a fixed set order. This is a constrained choice on development-used material. The accepted C1 model is not assembled by selecting a different condition for each evaluation family.